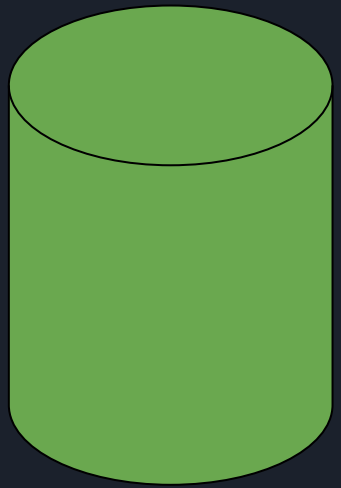
# GESTIÓN DE DATOS

## *ADMINISTRACIÓN: INDICES*

# RECORDERIS...



# ¿QUÉ APRENDEREMOS HOY?



- Qué es un índice
- Tipos de índices
- Concepto: Cardinalidad
- Cuando crearlos, cuando no
- Pros y contras

# PARA REFLEXIONAR (1 / 3)

¿Estaría bien recorrer 100 millones de registros para devolver sólo un registro con un dato puntual?

(discusión verbal)

# PARA REFLEXIONAR (2 / 3)

¿Que es más rápido y eficiente para buscar un tema puntual en un libro de 500 páginas?

(1) Buscar página por página hasta llegar al tema
(2) Leer la tabla de contenido e ir directo a la página

(discusión verbal)

# PARA REFLEXIONAR (3 / 3)

Si se usa una tabla de contenido para una revista de 10 páginas…

(1) Haría más rápida la búsqueda?
(2) O por el contrario, agrega complejidad?

(discusión verbal)

# ¡¡¡RECORDERIS!!!

**¡¡¡Conversemos de estructuras de datos!!!**

Recordemos la definición de:

- Lista enlazada y doblemente enlazada
- Matriz o array multidimensional
- Tabla hash o diccionario
- Árboles: B, B+, AVL, R, BST (binarios de búsqueda)
- Grafo



# ¡¡¡RECORDERIS!!!

**SPOILER:**

**Si no está claro cómo funcionan algunas estructuras de datos, no se entenderá como funciona un índice**

# CONCEPTO: CARDINALIDAD



# CONCEPTO: CARDINALIDAD

| id | full_name | genre | gender | height |
|---|---|---|---|---|
| 1 | SELENA GOMEZ | POP | FEMALE | 1.65 |
| 2 | BECKY G | REGGAETON | FEMALE | 1.52 |
| 3 | KAROL G | REGGAETON | FEMALE | 1.7 |
| 4 | TAYLOR SWIFT | POP | FEMALE | 1.8 |
| 5 | ARIANA GRANDE | POP | FEMALE | 1.53 |
| 6 | DUA LIPA | POP | FEMALE | 1.65 |
| 7 | CAMILA CABELLO | POP | FEMALE | 1.6 |
| 8 | SHAWN MENDES | POP | MALE | 1.88 |
| 9 | BILLIE EILISH | POP | FEMALE | 1.61 |
| 10 | ED SHEERAN | POP | MALE | 1.73 |
| 11 | JUSTIN BIEBER | POP | MALE | 1.75 |
| 12 | RIHANNA FENTY | R&B | FEMALE | 1.73 |
| 13 | KATY PERRY | POP | FEMALE | 1.7 |
| 14 | HARRY STYLES | POP | MALE | 1.8 |
| 15 | THE WEEKND TESFAYE | R&B | MALE | 1.78 |
| 16 | BEYONCE KNOWLES | R&B | FEMALE | 1.69 |
| 17 | ZAYN MALIK | POP | MALE | 1.75 |
| 18 | CARDI B | HIP HOP | FEMALE | 1.6 |
| 19 | POST MALONE | HIP HOP | MALE | 1.84 |
| 20 | LIZZO JEFFERSON | POP | FEMALE | 1.78 |

Cardinalidad por columna:

id: 20

full_name: 20

genre: 4

gender: 2

height: 13

# CONCEPTO: CARDINALIDAD

**Cantidad de valores distintos en un conjunto**

**….o también…**

**Cuántas veces aparece un valor en un conjunto**

# CONCEPTO: SELECTIVIDAD



# CONCEPTO: SELECTIVIDAD

| id | full_name | genre | gender | height |
|---|---|---|---|---|
| 1 | SELENA GOMEZ | POP | FEMALE | 1.65 |
| 2 | BECKY G | REGGAETON | FEMALE | 1.52 |
| 3 | KAROL G | REGGAETON | FEMALE | 1.7 |
| 4 | TAYLOR SWIFT | POP | FEMALE | 1.8 |
| 5 | ARIANA GRANDE | POP | FEMALE | 1.53 |
| 6 | DUA LIPA | POP | FEMALE | 1.65 |
| 7 | CAMILA CABELLO | POP | FEMALE | 1.6 |
| 8 | SHAWN MENDES | POP | MALE | 1.88 |
| 9 | BILLIE EILISH | POP | FEMALE | 1.61 |
| 10 | ED SHEERAN | POP | MALE | 1.73 |
| 11 | JUSTIN BIEBER | POP | MALE | 1.75 |
| 12 | RIHANNA FENTY | R&B | FEMALE | 1.73 |
| 13 | KATY PERRY | POP | FEMALE | 1.7 |
| 14 | HARRY STYLES | POP | MALE | 1.8 |
| 15 | THE WEEKND TESFAYE | R&B | MALE | 1.78 |
| 16 | BEYONCE KNOWLES | R&B | FEMALE | 1.69 |
| 17 | ZAYN MALIK | POP | MALE | 1.75 |
| 18 | CARDI B | HIP HOP | FEMALE | 1.6 |
| 19 | POST MALONE | HIP HOP | MALE | 1.84 |
| 20 | LIZZO JEFFERSON | POP | FEMALE | 1.78 |

Selectividad por columna:

id: Muy alta (20 valores únicos entre 20)

full_name: Muy alta (igual que id)

genre: Baja (4 valores entre 20)

gender: Muy baja (2 valores entre 20)

height: Alta (Pocos valores repetidos, 13 entre 20)

# CONCEPTO: SELECTIVIDAD

**Indica que tanto se puede reducir la cantidad de registros a buscar dependiendo de una condición**

✅✅✅ **Si reduce de 100'000.000 de registros a 1**

Ejemplo: `where person_id = '123'`

➖➖➖ **Si reduce de 100'000.000 a 50'000.000**

Ejemplo: `where gender = 'FEMALE'` **(asumiendo 50/50)**

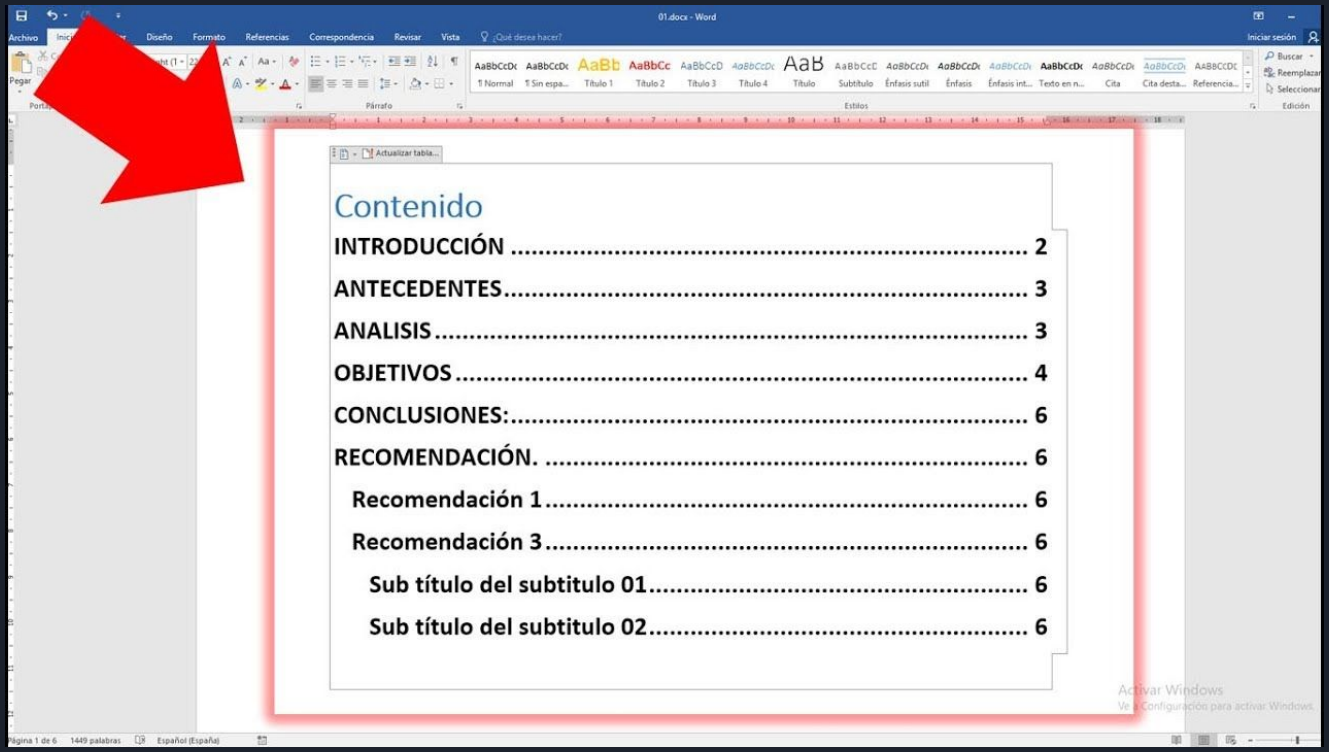
❌❌❌ **Si reduce de 100'000.000 a 99'000.000**

Ejemplo: `where age between 1 and 90`

# DEFINICIÓN: ÍNDICE



# analogía: ÍNDICE



# DEFINICIÓN: ÍNDICE

**Estructura de datos que permite agilizar las consultas evitando la lectura de datos innecesarios.**

**Nota adicional: Generalmente son árboles B+**

**Spoiler: "agilizar" no significa "hacer magia".**

# Definición: ÍNDICE INVERTIDO



# Definición: ÍNDICE INVERTIDO

Ejemplo de índice "***normal***":

```
SELENA GOMEZ..... 1
BECKY G.......... 2
KAROL G.......... 3
SELENA QUINTANILLA... 4
```

Ejemplo de índice ***invertido***:

```
SELENA..... [1, 4]
GOMEZ...... [1]
BECKY...... [2]
KAROL...... [2]
G.......... [2, 3]
QUINTANILLA ... [4]
```

Analogía:

Índice normal: indica el lugar de un valor completo
Índice invertido: indica el lugar de una palabra (para textos largos)

Spoiler: elasticseach, motores de búsqueda web, etc, usan índices invertidos

# Definición: ÍNDICE INVERTIDO

Ejemplo de índice "normal":

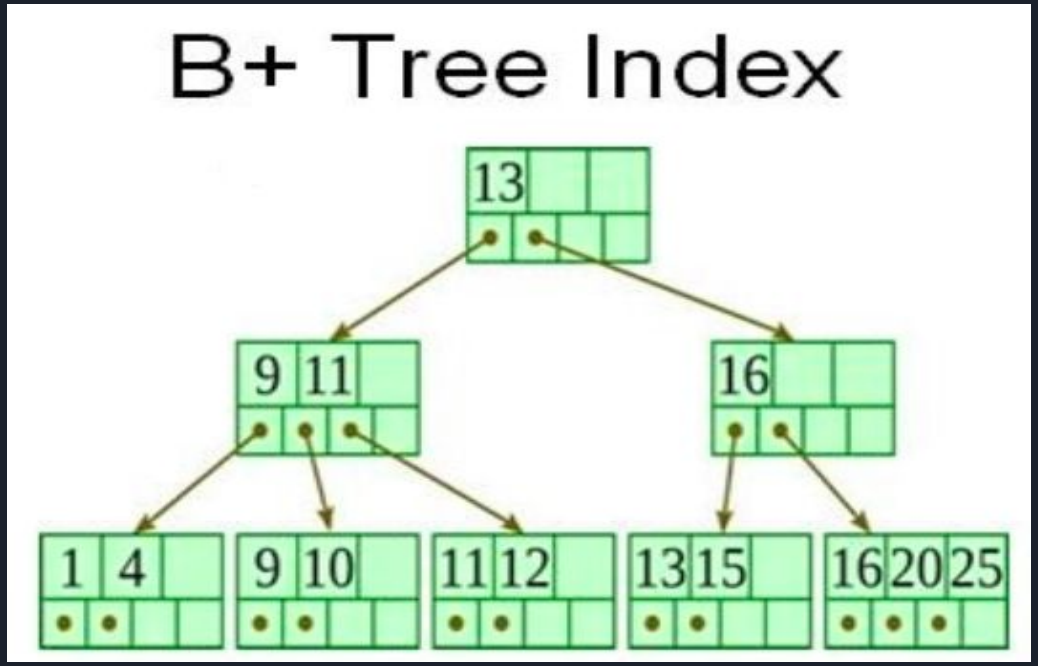
```
introducción a sql.........7
tipos de datos en sql.......15
sql y análisis de datos.....50
agregación con sql.........55
tipos de joins en sql.......60
```

Ejemplo de índice invertido:

```
introduccion... [7]
sql............ [7, 15, 50, 55, 60]
tipos.......... [15, 60]
analisis....... [50]
datos.......... [15, 50]
Agregacion..... [55]
joins.......... [60]
```

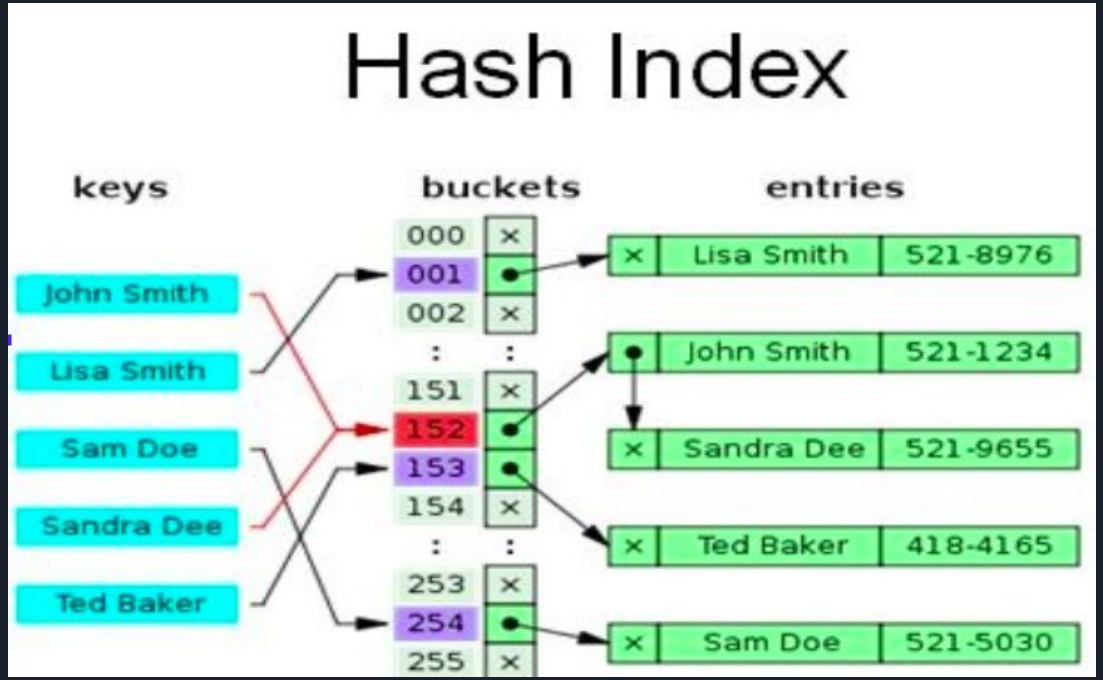
Spoiler: elasticsearch, motores de búsqueda web, etc, usan índices invertidos

# ÍNDICES: BTREE +



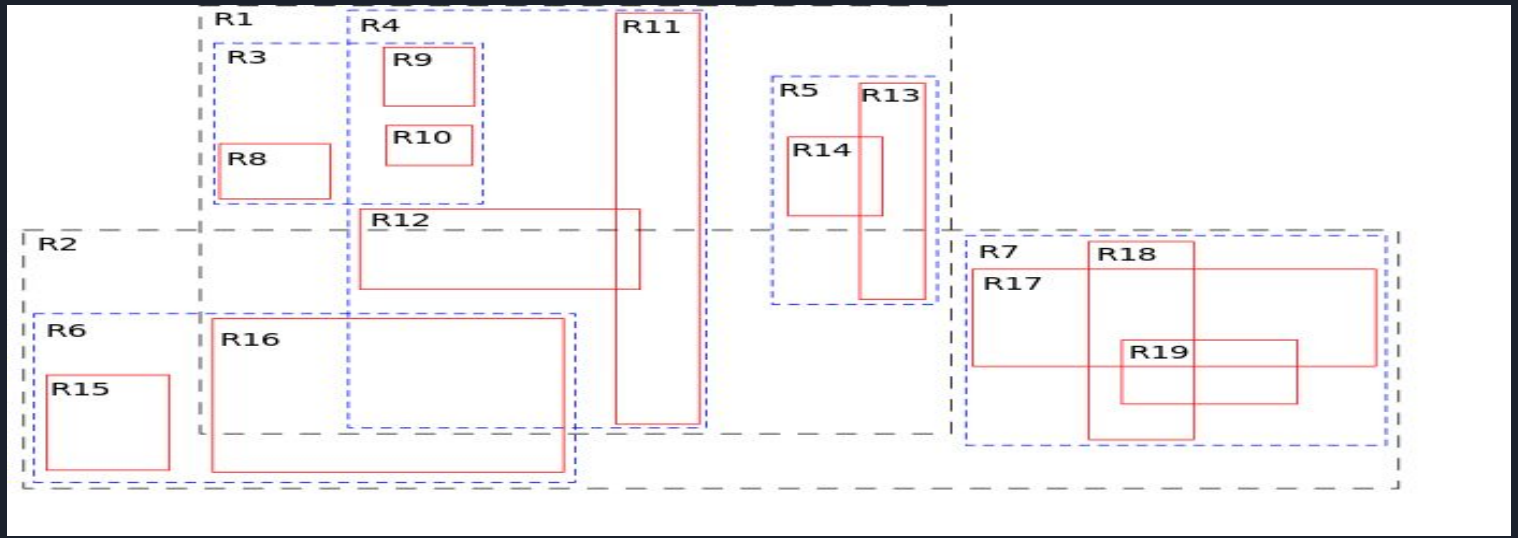

https://blogs.oracle.com/timesten/what-is-the-best-timesten-index-for-my-oltp-application

# ÍNDICES: HASH




https://blogs.oracle.com/timesten/what-is-the-best-timesten-index-for-my-oltp-application

# ÍNDICES: RTREE




https://i.stack.imgur.com/eVSp1.png

# ÍNDICES: Tipos por estructura

| Tipo de índice | Estructura de datos | Descripción |
|---|---|---|
| BTREE + | Árbol B+ y lista enlazada | Para búsquedas con rangos, usando operadores como >, >=, =, <, <=, BETWEEN, LIKE 'abc%'<br>Es el usado por defecto en la mayoría de dbms |
| HASH | Tabla hash | Para búsquedas con valores exactos.<br>No soporta rangos. |
| RTREE | Jerárquica con rectángulos | Para búsquedas geoespaciales.<br>Se usa en casos muy puntuales |
| Invertido | Tabla hash + lista | Para búsquedas de palabras sobre textos muy grandes |

# ÍNDICES: Tipos por valores

| Tipo | ¿duplicados? | ¿ nulos? | Descripción |
|---|---|---|---|
| Primary key | ❌ | ❌ | Garantizan que un registro sea único |
| Unique | ❌ | ✅ | Garantizan que un registro sea único (permitiendo valores nulos) |
| Plain | ✅ | ✅ | Permite valores duplicados |
| Full text | ✅ | ✅ | Para búsquedas de texto por palabras o frases completas en campos char, varchar y text |

# ÍNDICES DE 1 NIVEL

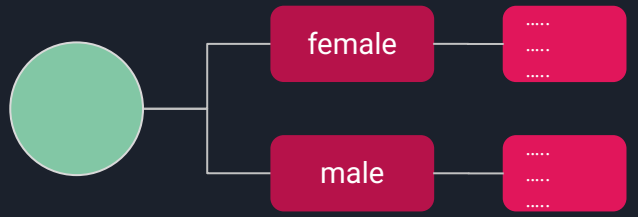
Suponer una tabla muy grande de clientes en la cual hay personas de varios países y la distribución entre mujeres y hombres es aproximadamente 50-50.

```
select *
from   people
where  genre = 'female'
and    id_type = 'CC'
```

Sin índice



Con índice



# ÍNDICES DE 1 NIVEL

Suponer una tabla muy g… …ay personas de varios países y la distribución … …aproximadamente 50-50.

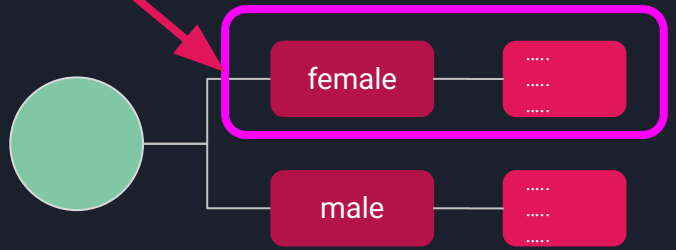
**OJO: Filtro del 50% de los datos a buscar**

```
select *
from   peopl…
where  genre = 'female'
and    id_type = 'CC'
```

Sin índice

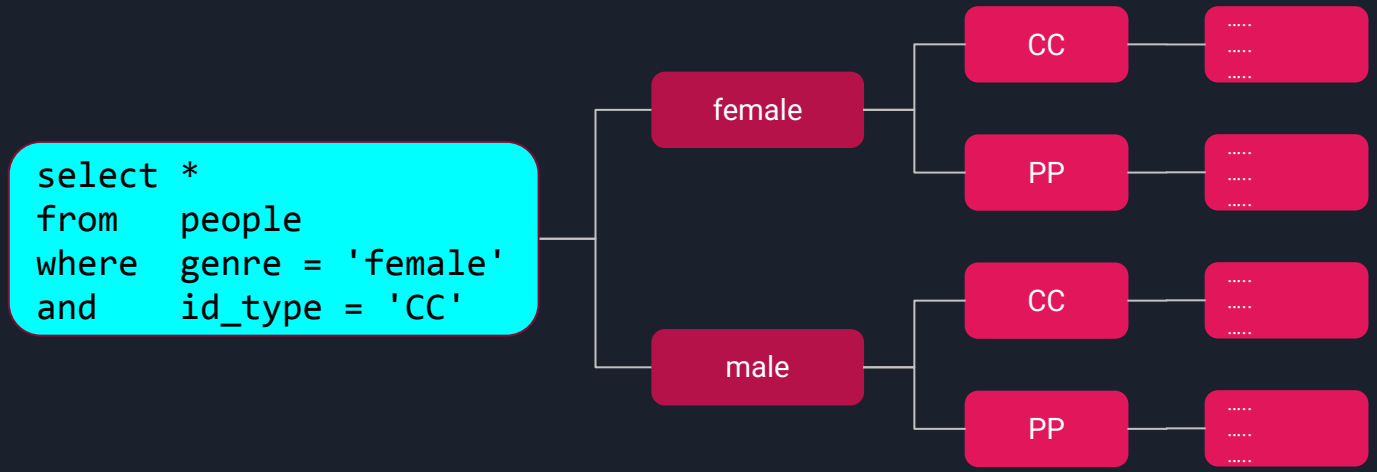


Con índice



# ÍNDICES DE "n" NIVELES

Como ejemplo, es posible indexar por género y luego por tipo de id. De esta forma se reducirá aún más la búsqueda en la tabla

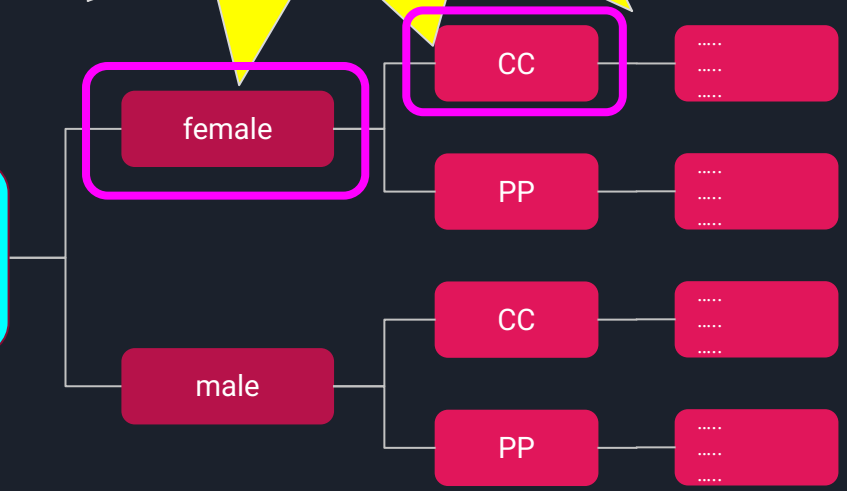


# ÍNDICES DE "n" NIVELES

Como ejemplo, es posible [illegible] por tipo de id. De esta forma se reducirá aún [illegible]

**Doble filtro:**
**Se omiten las búsquedas de "genre=male" y de "id_type" <> CC**

```
select *
from   people
where  genre = 'female'
and    id_type = 'CC'
```



# DEFINICIÓN: "Full Scan"

"Full scan" se refiere a ***recorrer la tabla entera*** para encontrar los registros que coincidan con una condición.

En tablas pequeñas:

- No tiene impacto significativo
- No aportará a mejorar el rendimiento

En tablas grandes:

- Afecta ***demasiado*** el rendimiento de las consultas
- Afecta el desempeño de las capas de software superiores

# RECORDANDO...

¿Que es más rápido y eficiente para buscar un tema puntual en un libro de 500 páginas?

(1) Buscar página por página hasta llegar al tema
=> ESTO ES FULL SCAN

(2) Leer la tabla de contenido e ir directo a la página
=> ESTO ES USO DE ÍNDICES

# IMPORTANTE!!!

No hay una definición exacta de qué puede ser una "tabla pequeña" y una "tabla grande".

Generalmente una tabla de 1.000 registros es muy pequeña; cientos de miles de registros, mediana; y decenas de millones de registros tiende a ser grande.

Una operación de lectura intensiva sobre tablas grandes consumirá mucha cpu y lectura innecesaria en disco, salvo que haya índices y estén creados correctamente.



# IMPORTANTE!!!

***El "full scan" es uno de los peores enemigos del rendimiento en cualquier dbms***, tanto para la capa de datos como para las capas superiores, por la contención que genera y por el exceso de lectura en el medio de almacenamiento.

La clave para evitarlo es:

- Definir correctamente los índices a crear
- Escribir consultas sql con filtros bien hechos en el "where"



# ÍNDICES: ¿Qué tener en cuenta?

- Consulta más rápida → Escritura más lenta
  - Puede implicar rebalanceo de los árboles en insert, update, delete.
- No todo es candidato a indexar
  - Identificar únicamente los campos que necesiten ser indexados!!!
- El operador "like" puede anular el uso de índices
  - `where nombre like 'A%'` → Usa índice
  - `where nombre like '%A'` → No usa índice ⇒ Ojo con esto!!!
  - `where nombre like '%A%'` → No usa índice
- En los índices multinivel, ***el orden importa y mucho***
  - Ej: No es lo mismo indexar por sexo biológico (nivel 1) y luego por profesión (nivel 2), que al contrario (discutir con ejemplo gráfico).

# ÍNDICES: ¿Qué tener en cuenta?

- Indexar con sabiduría!
  - Discusión grupal: Asumir que la tabla "singers" tiene 10 millones de registros. Plantear búsquedas por cada campo y concluir si vale la pena o no indexar.

```
MariaDB [singers]> desc singers;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| id         | int(11)     | NO   | PRI | 0       |       |
| id_type    | varchar(3)  | NO   | PRI | NULL    |       |
| first_name | varchar(30) | NO   |     | NULL    |       |
| last_name  | varchar(30) | YES  |     | NULL    |       |
| birth_date | date        | NO   |     | NULL    |       |
| genre      | varchar(15) | NO   | MUL | NULL    |       |
+------------+-------------+------+-----+---------+-------+
6 rows in set (0,028 sec)
```

# ÍNDICES: ¿Como se crean?

**SPOILER**

**Crear índices es muy fácil, no tiene ciencia**

**Lo importante es crear el índice *correcto***

# ÍNDICES: ¿Como se crean?

**Opción 1:**

La tabla ya existe y se va a crear un índice ***plano*** tipo btree en un campo.

NOTA: Es posible especificarle un nombre al índice.

```
alter table singers add index (first_name) using btree;

alter table singers add index idx_sing_fina(first_name) using btree;

create index idx_sing_fina on singers(first_name) using btree;
```

# ÍNDICES: ¿Como se crean?

**Opción 2:**

La tabla ya existe y se va a crear un índice ***único*** tipo btree en un campo.

```
alter table singers add unique index (first_name) using btree;

alter table singers add unique index idx_sing_fina(first_name) using btree;

create unique index idx_sing_fina on singers(first_name) using btree;
```

# ÍNDICES: ¿Como se crean?

**Opción 3:**

La tabla ya existe y se va a crear un índice de ***texto completo*** en un campo.

```
alter table singers add fulltext index (first_name);

alter table singers add fulltext index idx_sing_fina(first_name);

create fulltext index idx_sing_fina on singers(first_name);
```

# ÍNDICES: ¿Como se crean?

**Opción 4:**

La tabla no existe y se va a crear especificando un índice tipo clave primaria (es el caso más común).

```
create table singers (
    id int(11) not null,
    first_name varchar(30) not null,
    primary key (id)  ⇒ La clave primaria por defecto tiene índice
);
```

# ÍNDICES: ¿Como se crean?

**Opción 5:**

La tabla no existe y se va a crear especificando un índice tipo clave primaria y adicionalmente un índice tipo único.

```
create table singers (
    id int(11) not null,
    first_name varchar(30) not null,
    primary key (id),
    unique (first_name) ⇒ Índice tipo único
);
```

# ÍNDICES: ¿Como se crean?

**Opción 6:**

La tabla no existe y se va a crear especificando un índice tipo clave primaria y adicionalmente un índice tipo texto completo.

```
create table singers (
    id int(11) not null,
    first_name varchar(30) not null,
    primary key (id),
    fulltext (first_name) ⇒ Índice tipo texto completo
);
```

# ÍNDICES: ¿Como se crean?

**Opción 7:**

índice plano multinivel (indexar varios campos en orden)

```
create table singers (
    id int(11) not null,
    first_name varchar(30) not null,
    last_name varchar(30) not null,
    primary key (id),
    index (last_name, first_name) ⇒ Nivel 1: last_name. Nivel 2: first_name
);

create index idx_sing_lana_fina on singers(last_name, first_name);
```

# IMPORTANTE!!!

Si al momento de crear un índice plano, único o clave primaria, no se especifica el tipo (ej: using btree, using hash), MariaDB y MySQL usarán por defecto el tipo de índice BTREE.

Los índices fulltext no necesitan especificar la estructura de datos ya que internamente usan índices invertidos.



# ÍNDICES: ¿Como se borran?

Se debe conocer el nombre del índice, o en su defecto especificar el campo.

Se puede con "alter table" o con "drop index".

```
alter table singers drop index first_name;

alter table singers drop index idx_sing_fina;

drop index idx_sing_fina on singers;
```

# ÍNDICES: ¿Cuales existen?

**Se muestra** información de todos los índices de una tabla. Esto incluye tipo de índice por estructura y por valor, campos indexados, cardinalidad, entre otros.

Es posible aplicar filtros a la consulta.

```
show indexes from singers;

show indexes from singers where Column_name = 'id';
```

# IMPORTANTE!!!

MariaDB permite crear varios índice para un mismo campo.

Tener en cuenta que esto impactará el rendimiento y no aportará a mejorar los tiempos de respuesta!



# ÍNDICES: discusión grupal

Que clase de índices (por estructura y datos) usar para:



- Tabla de clientes con id, tipo_id, nombre, apellidos, género, dirección, ciudad, país
- Tabla de históricos de inventario generada por un trigger
- Tabla de inventario
- Registro de ingreso a la universidad usando los torniquetes
- Tienda de ropa en línea

# ÍNDICES: discusión grupal

- ¿Cual es el peor índice que se podría crear en esta tabla?

```
MariaDB [singers]> desc singers;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| id         | int(11)     | NO   | PRI | 0       |       |
| id_type    | varchar(3)  | NO   | PRI | NULL    |       |
| first_name | varchar(30) | NO   |     | NULL    |       |
| last_name  | varchar(30) | YES  |     | NULL    |       |
| birth_date | date        | NO   |     | NULL    |       |
| genre      | varchar(15) | NO   | MUL | NULL    |       |
+------------+-------------+------+-----+---------+-------+
6 rows in set (0,028 sec)
```

# EJERCICIOS

Preparación

Importar la base de datos "tienda", ubicada en uvirtual, en una instancia MariaDB.
Asegurarse que queden creadas las tablas:

- Usuarios, con 500.000 registros
- Productos, con 50.000 registros
- Compras, con 15'000.000 registros

***NOTA: Importar una base de datos desde un script sql es algo que a esta altura ya usted debe conocer, por lo cual se pasan por alto los pasos para indicar cómo se hace.***

# EJERCICIOS

Ejercicio 1

Escribir sentencia sql para contar las compras con método de pago "CREDIT_CARD" y tomar nota del resultado y del tiempo de respuesta.

Ejercicio 2

Escribir sentencia sql para contar las compras de las fechas octubre 26 de 2020, mayo 31 de 2020, 9 de enero de 2024. Tomar nota del resultado y el tiempo de respuesta.

# EJERCICIOS

Ejercicio 3

Escribir sentencia sql para contar las compras realizadas en efectivo únicamente por mujeres. Tomar nota del resultado y del tiempo de respuesta.

Ejercicio 4

Escribir sentencia sql para contar las compras realizadas en el año 2021 de los productos marca Samsung y Apple. Tomar nota del resultado y del tiempo de respuesta.

# EJERCICIOS

Ejercicio 5

Para los ejercicios 1 y 2, identificar el tipo de índice más óptimo para mejorar los tiempos de respuesta y justificarlo. Crearlo, repetir las sentencias sql y comparar los tiempos de respuesta.

Ejercicio 6

Para los ejercicios 3 y 4, identificar qué índice o índices crear en cuales tablas para mejorar los tiempos de respuesta. Crearlo (o crearlos), repetir las sentencias sql y comparar los tiempos de respuesta.

# EJERCICIOS

Ejercicio 7

Sacar conclusiones respecto a todos los ejercicios anteriores

# ¿PREGUNTAS?

